

GitLab MCP Security Report

Vulnerability Assessment & Penetration Testing

SAST / SCA / DAST / OWASP & MCP Security Analysis

Target: GitLab MCP Server (awgment-gitlab-mcpserver-dev) v2.1.0

Confidential

Table of Contents

Executive Summary.....	3
Summary of All Findings.....	3
Snapshot of All Findings and Current Status	4
Introduction.....	5
Findings.....	6
Test Coverage and Mapping Results to Standards	6
Vulnerabilities Details.....	8
Limitations and Exclusions.....	25
Best Practices.....	25
Conclusion	25
Disclaimer.....	26

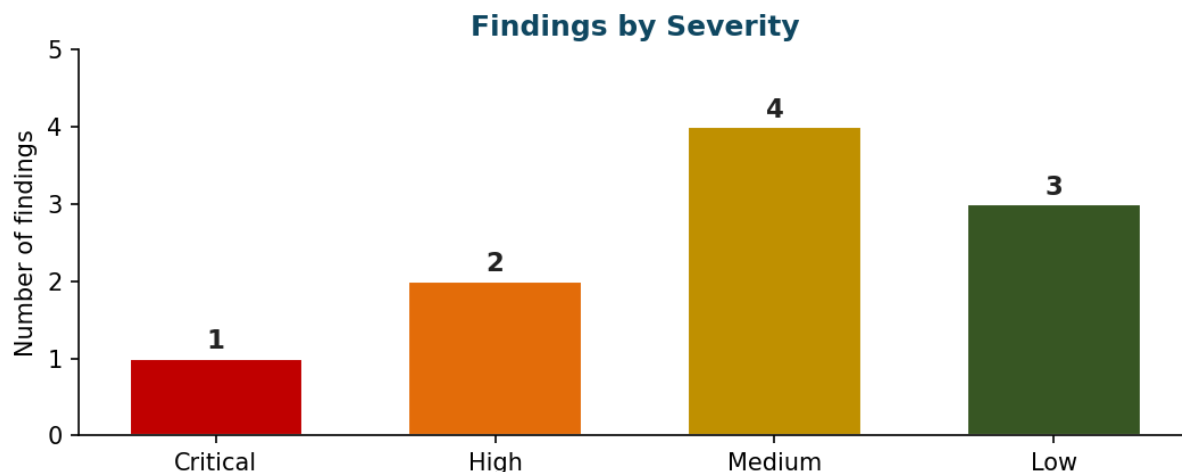
Executive Summary

This section gives a comprehensive summary of the VAPT engagement, highlighting its main findings and the general security status of the GitLab MCP Server's security posture.

Engagement Date	September 2026
Target Systems	GitLab MCP Server (awgmont-gitlab-mcpserver-dev) v2.1.0
Scope	The MCP server production code (app/), its dependency tree (pyproject.toml, uv.lock), container and CI manifests (Dockerfile, .github/workflows/ci.yml), and the exposed MCP tool and transport surface (stdio, TCP, HTTP).
Methodology	This VAPT is conducted following the NIST framework, the OWASP API Security Top 10 (2023), the OWASP MCP Top 10 (2025), and industry best practices. Techniques: static analysis (Bandit + Semgrep + manual review), software composition analysis (pip-audit against the locked dependency tree), and MCP transport/tool security review.
Overall Security Posture	The GitLab MCP Server carries one Critical and two High findings. The most severe are an unauthenticated HTTP MCP endpoint exposed on 0.0.0.0:8001 by default, destructive tools that execute with no human-in-the-loop approval, and raw request headers (containing caller credentials) logged at WARNING level. Overall risk is High and is deployment- and transport-mode dependent; the stdio-only mode is materially lower risk.
Key Findings	Refer to the table below for the summary of vulnerabilities found, severity, and current status. A total of 10 findings were identified (1 Critical, 2 High, 4 Medium, and 3 Low).
Recommendations	Plan to fix the Critical/High severity vulnerabilities reported at the earliest, then address the Medium and Low hardening items and re-test.

Summary of All Findings

The below chart summarizes all identified vulnerabilities and their current status:



Snapshot of All Findings and Current Status

The table below summarizes all identified vulnerabilities and their current status.

V-ID	Name of the Vulnerability	Severity	Current Status
1	Unauthenticated HTTP MCP endpoint with no endpoint auth or TLS	Critical	OPEN
2	Destructive MCP tools lack human-in-the-loop / approval gates	High	OPEN
3	Raw HTTP headers (with caller credentials) logged at WARNING level	High	OPEN
4	TCP transport framing missing max_message_size (unbounded allocation)	Medium	OPEN
5	Stdio transport lacks application-layer authentication	Medium	OPEN
6	Unbounded resource consumption / no rate limiting	Medium	OPEN
7	Dynamic URL path segments not URL-encoded in many services	Medium	OPEN
8	Generic exception strings returned to MCP clients	Low	OPEN
9	Tool execution audit hook defined but not wired into the registry	Low	OPEN
10	Verbose validation errors disclose framework version and internal model names	Low	OPEN

Introduction

This section gives additional information regarding the purpose and context of the GitLab MCP Server VAPT.

Purpose of the Engagement

To detect security weaknesses in the GitLab MCP Server by utilizing static analysis, software composition analysis, and manual review methods, thus evaluating its strength against actual attacks.

Target - GitLab MCP Server

The target is a Python 3.11+ MCP server (aiohttp, Pydantic v2) that exposes GitLab API operations as Model Context Protocol tools over stdio, TCP, and Streamable HTTP transports.

Scope of the Engagement

In-Scope: The MCP server production code (app/), its dependency tree (pyproject.toml, uv.lock), container and CI manifests (Dockerfile, .github/workflows/ci.yml), and the exposed MCP tool and transport surface.

Out-of-Scope: The upstream GitLab instance itself, third-party backend datastores, and the underlying host/cluster infrastructure beyond the provided manifests. Dynamic testing (DAST), network penetration testing, live GitLab instance testing, and the .env file / live credentials were excluded.

Methodology Followed

The process included: This VAPT employed a systematic method in accordance with the NIST framework and OWASP security risks. The procedure encompassed:

- Information Gathering and Reconnaissance
- Static Analysis (SAST) and Dependency Analysis (SCA)
- MCP Transport and Tool Security Review
- Reporting and Recommendations

Findings

This is the main report, outlining every identified vulnerability. It is designed for technical stakeholders.

Test Coverage and Mapping Results to Standards

The tables below illustrate the security posture of the GitLab MCP Server against the OWASP lists applicable to this MCP server: the OWASP API Security Top 10 (2023) and the OWASP MCP Top 10 (2025). For each category, a status of Fail indicates that one or more findings were identified, and Pass indicates that the category was assessed with no findings. Categories that are not applicable to this codebase are marked N/A.

OWASP API Security Top 10 (2023)

S.N o.	Vulnerability	Status	Reference Vulnerabilities
1	API1:2023 Broken Object Level Authorization	Pass	Pydantic validation and optional GITLAB_ALLOWED_PROJECTS allowlist; no BOLA path found.
2	API2:2023 Broken Authentication	Fail	V1, V5
3	API3:2023 Broken Object Property Level Authorization	Pass	Inputs bound to Pydantic *Input models; no mass-assignment pattern found.
4	API4:2023 Unrestricted Resource Consumption	Fail	V4, V6
5	API5:2023 Broken Function Level Authorization	Fail	V2
6	API6:2023 Unrestricted Access to Sensitive Business Flows	Pass	No high-value business flows exposed through the API.
7	API7:2023 Server Side Request Forgery	Pass	Outbound requests constrained to configured GITLAB_URL; no user-controlled URL host/scheme.
8	API8:2023 Security Misconfiguration	V1, V3, V8, V10	V1, V3, V8
9	API9:2023 Improper Inventory Management	Fail	V9
10	API10:2023 Unsafe Consumption of APIs	Pass	Path segments encoded with quote(); JSON parsed safely; structured errors propagated.

OWASP MCP Top 10 (2025)

S.N o.	Vulnerability	Status	Reference Vulnerabilities
1	MCP01:2025 Token Mismanagement & Secret Exposure	Fail	V3
2	MCP02:2025 Privilege Escalation via Scope Creep	Fail	V1, V2
3	MCP03:2025 Tool Poisoning	Pass	Static tool definitions registered at startup; no rug-pull / mutable definitions found.

4	MCP04:2025 Software Supply Chain Attacks & Dependency Tampering	Pass	uv.lock present and current; pip-audit found 0 CVEs; base image digest-pinned.
5	MCP05:2025 Command Injection & Execution	Pass	No eval / exec / subprocess / pickle / yaml.load / os.system sinks in app/.
6	MCP06:2025 Intent Flow Subversion	Pass	GitLab-sourced content is prefixed with an advisory untrusted-content warning banner; no confirmed intent-subversion finding in scope.
7	MCP07:2025 Insufficient Authentication & Authorization	Fail	V1, V2, V5
8	MCP08:2025 Lack of Audit and Telemetry	Fail	V9
9	MCP09:2025 Shadow MCP Servers	Pass	Single server instance; no dynamic tool loading from other servers/registries.
10	MCP10:2025 Context Injection & Over-Sharing	Pass	Variable reveal and job-trace secrets sit behind default-deny redaction gates; no confirmed over-sharing finding in scope.

Vulnerabilities Details

Below are the specific details for each identified vulnerability, ordered from Critical to Informational.

V-ID 1: Unauthenticated HTTP MCP endpoint with no endpoint auth or TLS

V-ID	1
Name of the Vulnerability	Unauthenticated HTTP MCP endpoint with no endpoint auth or TLS
Risk	Critical Reference: 9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Likelihood	High
Impact	High
Current Status	OPEN
Observation	The full transport mode (gitlab-mcp-full console script, app/main.py:run_full) sets the MCP transport to http and starts a Streamable HTTP listener in app/mcp/server.py:_run_http. The default host is 0.0.0.0 and the default port is 8001 (app/config/settings.py:142-147). The aiohttp application accepts POST requests carrying MCP JSON-RPC calls (tools/call, tools/list, etc.) and a /health endpoint, but it does not enforce any API-key, HTTP Basic/Digest auth, bearer token, mTLS, origin validation, or session management on the MCP endpoint itself. The only caller-authentication mechanism is extraction of a GitLab PAT from inbound request headers (used for outbound GitLab API calls), but that PAT is not validated as a credential for the MCP endpoint. Without a reverse proxy or sidecar enforcing authentication and TLS, a network-reachable deployment allows anyone to invoke any registered tool with the full scope of the configured PAT. Standards Mapping: OWASP API2:2023 Broken Authentication, API8:2023 Security Misconfiguration OWASP MCP07:2025 Insufficient Authentication & Authorization, MCP02:2025 Privilege Escalation via Scope Creep CWE-306, CWE-319 NIST SP 800-53 IA-2, SC-8, AC-3 SOC 2 CC6.1, CC6.6, CC7.2
Risk/Business Impact	An unauthenticated attacker who can reach the HTTP port can list all tools, call destructive tools (delete_project, merge_merge_request, block_user, erase_job, delete_variable, etc.), read CI variables (if GITLAB_ALLOW_VARIABLE_REVEAL=true), and exfiltrate data from the GitLab instance at the scope of the configured PAT. When combined with LLM-driven access, this creates an excessive-agency risk where an attacker can instruct the model to invoke tools with no further approval.

Recommendation	Add an authentication and transport-security layer before <code>handle_mcp_post</code> is reached. At minimum, deploy behind an authenticated TLS reverse proxy and bind the server to 127.0.0.1 by default. In code, add a <code>check_auth</code> middleware and an <code>enforce_https</code> helper, make the default host loopback-only unless explicitly overridden, and reject HTTP requests in production when the incoming connection is not TLS-secured. Also consider <code>Mcp-Session-Id</code> validation, origin checking, and mTLS for production. Secure code example: <pre># app/config/settings.py http_host: str = Field(default="127.0.0.1", description="HTTP host for MCP server") http_require_auth: bool = Field(default=True, description="Require API key on HTTP endpoint") api_key: str = Field(default="", description="MCP endpoint API key (or mTLS)") # app/mcp/server.py -- inside _run_http async def require_api_key(request: "web.Request", handler): if not self.settings.mcp.http_require_auth: return await handler(request) if not request.headers.get("Authorization", "").startswith("Bearer "): raise web.HTTPUnauthorized(text="MCP endpoint requires a valid API key") token = request.headers["Authorization"][7:] if not hmac.compare_digest(token, self.settings.mcp.api_key): raise web.HTTPUnauthorized(text="Invalid API key") return await handler(request) app = web.Application(client_max_size=max_bytes, middlewares=[require_api_key])</pre>
Impacted Assets	app/main.py:131-141 app/mcp/server.py:391-490 (HTTP transport setup) app/mcp/server.py:845-876 (header extraction for caller PAT) app/config/settings.py:142-147

Proof Of Concept

```
# app/main.py:131-141
def run_full() -> None:
    logger.info("Running in full mode (MCP server over Streamable HTTP)")
    settings = get_settings()
    settings.mcp.transport = "http"
    asyncio.run(main(settings=settings))

# app/mcp/server.py:391-490 (excerpt)
async def _run_http(self) -> None:
    host = self.settings.mcp.http_host
    port = self.settings.mcp.http_port
    path = self.settings.mcp.http_path
    max_bytes = self.settings.mcp.max_message_size

    async def handle_mcp_post(request: "web.Request") -> "web.Response":
        # ... size check only ...
        raw_body = await request.read()
        payload = json.loads(raw_body.decode("utf-8"))
        result = await self.handle_jsonrpc_request(payload,
            headers=dict(request.headers))
        ...

    app = web.Application(client_max_size=max_bytes)
    app.router.add_post(path, handle_mcp_post)
    app.router.add_get(path, handle_mcp_get)
    app.router.add_get("/health", handle_health)

    runner = web.AppRunner(app)
    await runner.setup()
    site = web.TCPSite(runner, host=host, port=port)
    await site.start()

# app/config/settings.py:142-147
http_host: str = Field(default="0.0.0.0", description="HTTP host for MCP
server")
http_port: int = Field(default=8001, description="HTTP port for MCP
server")
http_path: str = Field(
    default="/api/awgument-eqms/gitlab", description="HTTP endpoint path
for MCP server"
)
```

V-ID 2: Destructive MCP tools lack human-in-the-loop / approval gates

V-ID	2
Name of the Vulnerability	Destructive MCP tools lack human-in-the-loop / approval gates
Risk	High Reference: 8.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:H) when combined with unauthenticated HTTP; 6.8 (CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:N/I:H/A:H) when stdio/PAT auth is present
Likelihood	High
Impact	High

Current Status	OPEN
Observation	A large number of destructive or high-impact tools (delete_file, delete_project, merge_merge_request, block_user, erase_job, delete_variable, delete_webhook, delete_group, close_merge_request, approve_merge_request, unapprove_merge_request, rebase_merge_request, create_variable, update_variable, cancel_pipeline, delete_pipeline, retry_pipeline, play_job, delete_branch, protect_branch, unprotect_branch, delete_tag, protect_tag, unprotect_tag, delete_release, delete_release_link, delete_label, delete_milestone, promote_milestone, delete_snippet, delete_wiki_page, test_webhook, etc.) are registered in register.py and can be invoked with a single tools/call JSON-RPC request. The base tool class (app/mcp/tools/gitlab/_base.py) enforces GITLAB_ALLOWED_PROJECTS and validates arguments via Pydantic, but there is no confirmation step, no second-factor approval, no risk-class flag, and no explicit human-in-the-loop guard. The category router validates the action name against a closed enum but still dispatches immediately. In LLM-driven deployments, the model can autonomously choose and execute destructive actions with the scope of the PAT. Standards Mapping: OWASP API5:2023 Broken Function Level Authorization, API2:2023 Broken Authentication OWASP MCP07:2025 Insufficient Authentication & Authorization, MCP02:2025 Privilege Escalation via Scope Creep CWE-250, CWE-284, CWE-306 NIST SP 800-53 AC-3, AC-6, CM-7 SOC 2 CC6.1, CC6.2, CC7.2
Risk/Business Impact	A compromised MCP client, a prompt-injection against the LLM, or an unauthenticated HTTP caller (V-ID 1) can delete repositories, merge unreviewed code, block users, erase CI job traces, and modify production secrets. The blast radius is limited by GitLab PAT scopes, but the MCP server itself does not enforce any separation between read-only and destructive actions beyond the tool name and schema.
Recommendation	Introduce a per-tool destructive flag and an explicit approval token that must be supplied by the caller (or an external approver) for any destructive or high-impact action. In the LLM client, surface the requested action to the user and require explicit confirmation before generating an approval token. For HTTP mode, require a separate authenticated /approve endpoint to mint tokens. Keep destructive tools disabled when the transport is unauthenticated. Secure code example: <pre># app/mcp/tools/gitlab/_base.py DESTRUCTIVE_TOOLS = {"delete_project", "merge_merge_request", "block_user", "erase_job", ...} def require_approval(tool_name: str, arguments: dict, approved_request_ids: set) -> bool: if tool_name not in DESTRUCTIVE_TOOLS: return True approval_token = arguments.pop("__approval_token", None) if approval_token and approval_token in approved_request_ids: return True return False</pre>

Impacted Assets	app/mcp/tools/gitlab/files.py:142-170 (delete_file) app/mcp/tools/gitlab/projects.py:158-178 (delete_project) app/mcp/tools/gitlab/merge_requests.py:165-188 (merge_merge_request) app/mcp/tools/gitlab/users.py:95-115 (block_user) app/mcp/tools/gitlab/jobs.py:195-221 (erase_job) app/mcp/tools/gitlab/variables.py:173-199 (delete_variable) app/mcp/tools/gitlab/webhooks.py:161-187 (delete_webhook) app/mcp/tools/gitlab/_base.py:82-153 (common execute wrapper) app/mcp/tools/gitlab/category_router.py:75-93 (dispatch) app/mcp/tools/gitlab/register.py:248-590 (registration of all tools)
Proof Of Concept	<pre> # app/mcp/tools/gitlab/projects.py:158-178 class DeleteProjectTool(GitLabTool): def __init__(self, api): self.api = api super().__init__(name="delete_project", description="Delete a GitLab project.", input_schema=DeleteProjectInput.model_json_schema(), output_schema={...}, error_label="delete project",) async def run(self, arguments): req = DeleteProjectInput(**arguments) await self.api.delete_project(req.project_id) return text_result("Project deleted successfully.") # app/mcp/tools/gitlab/_base.py:82-153 class GitLabTool(Tool): async def execute(self, arguments: Dict[str, Any]) -> ToolResult: # allowlist check, token override, then: result = await self.run(arguments) return result # No require_approval or destructive gate is present. </pre>

V-ID 3: Raw HTTP headers (with caller credentials) logged at WARNING level

V-ID	3
Name of the Vulnerability	Raw HTTP headers (with caller credentials) logged at WARNING level
Risk	High Reference: 7.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Likelihood	Medium
Impact	High
Current Status	OPEN

Observation	In app/mcp/protocol/stateless_handler.py:64, the stateless handler logs the entire inbound HTTP header dictionary at the WARNING level with the message 'DEBUG raw headers received: %s'. Because the server extracts caller credentials from headers such as PRIVATE-TOKEN, Authorization: Bearer ..., X-API-Key, and X-GitLab-Token, those secret values are frequently present in the logged header dict. Logging at WARNING means the values are written to the default application log output, log files, and log aggregators even when LOG_LEVEL is left at its default INFO value. This is a credential-disclosure sink regardless of the logging destination. Standards Mapping: OWASP API8:2023 Security Misconfiguration OWASP MCP01:2025 Token Mismanagement & Secret Exposure, MCP08:2025 Lack of Audit and Telemetry CWE-532 NIST SP 800-53 AU-3, AU-9, SC-28 SOC 2 CC7.2, CC6.1
Risk/Business Impact	An attacker or insider with read access to logs, stdout, stderr, or a log aggregation platform can harvest live GitLab PATs / bearer tokens from the MCP endpoint. Those tokens can be replayed directly against the GitLab instance or against the MCP server itself, leading to data exfiltration and destructive actions. This finding is independent of the transport authentication issue but compounds it by making credential theft easier.
Recommendation	Remove the raw header log line entirely. If header diagnostics are needed, log at DEBUG only after redacting credential-bearing keys. Never log the full values of Authorization, PRIVATE-TOKEN, X-API-Key, X-GitLab-Token, or Cookie. Secure code example: <pre># app/mcp/protocol/stateless_handler.py SENSITIVE_HEADERS = {"authorization", "private-token", "x-api-key", "x-gitlab-token", "cookie"} if self._server.logger.isEnabledFor(logging.DEBUG): safe_headers = { k: ("[REDACTED]" if k.lower() in SENSITIVE_HEADERS else v) for k, v in (headers or {}).items() } self._server.logger.debug("Stateless handler headers: %s", safe_headers) # Then delete the existing logger.warning(...) line.</pre>
Impacted Assets	app/mcp/protocol/stateless_handler.py:64
Proof Of Concept	<pre># app/mcp/protocol/stateless_handler.py:61-64 if method == "tools/call": self._server.logger.warning("DEBUG raw headers received: %s", headers) # Same per-user override mechanism as the 2025-11-25 stateful # path in MCPServer._route_message: pull the caller's GitLab # PAT out of the request headers ...</pre>

V-ID 4: TCP transport framing missing max_message_size (unbounded allocation)

V-ID	4
Name of the Vulnerability	TCP transport framing missing max_message_size (unbounded allocation)

Risk	Medium Reference: Medium-High (manual review, 2026-09-03) - unbounded memory allocation in TCP framing path
Likelihood	Low
Impact	High
Current Status	OPEN
Observation	The TCP framing loop in app/mcp/server.py:_handle_tcp_connection (lines 350-389) reads a 4-byte length and then calls reader.readexactly(message_length) without enforcing max_message_size. The _run_tcp and _run_websocket methods are referenced in start() but are never defined, so the TCP/WebSocket transport wiring is currently incomplete and the code is non-functional. However, if the transport were enabled, a remote client could request an arbitrary memory allocation and cause a denial of service. This is a concrete instance of unbounded resource consumption in the TCP transport path (related to V-ID 6). Standards Mapping: OWASP API4:2023 Unrestricted Resource Consumption CWE-770, CWE-400 NIST SP 800-53 SC-5, SC-6 SOC 2 CC6.1, CC7.1
Risk/Business Impact	If the TCP/WebSocket transport is completed and enabled, a remote client can send a large framed length prefix and force the server to allocate arbitrary memory before any size validation, exhausting host memory and denying service to legitimate users.
Recommendation	Enforce max_message_size on the TCP framing path before calling readexactly(): validate the decoded 4-byte length against the configured maximum and reject oversized frames. Complete the _run_tcp / _run_websocket wiring with the same body-size, timeout, and throttling controls used on the HTTP transport before enabling these transports in any environment.
Impacted Assets	app/mcp/server.py:350-389 (_handle_tcp_connection) app/mcp/server.py (start(): _run_tcp and _run_websocket referenced but never defined)
Proof Of Concept	<pre># app/mcp/server.py:_handle_tcp_connection (350-389, excerpt) # reads a 4-byte length prefix, then: message_length = int.from_bytes(header, "big") payload = await reader.readexactly(message_length) # no max_message_size check # _run_tcp and _run_websocket are referenced in start() but never defined.</pre>

V-ID 5: Stdio transport lacks application-layer authentication

V-ID	5
Name of the Vulnerability	Stdio transport lacks application-layer authentication
Risk	Medium Reference: 5.3 (CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)
Likelihood	Low
Impact	High
Current Status	OPEN

Observation	The minimal entry point (gitlab-mcp-minimal, app/main.py:run_minimal) sets the MCP transport to stdio and starts the server in app/mcp/server.py:_run_stdio. The loop reads JSON-RPC lines from sys.stdin, parses them, and dispatches them to the tool registry. There is no client identity check, token, or session binding. Any process or user that can attach to the MCP process's standard streams can invoke any registered tool with the scope of the configured GitLab PAT. This is the standard MCP stdio model, but it becomes a trust-boundary issue on shared hosts, multi-user containers, or misconfigured environments where stdin/stdout are accessible to other processes. Standards Mapping: OWASP API2:2023 Broken Authentication OWASP MCP07:2025 Insufficient Authentication & Authorization CWE-306 NIST SP 800-53 IA-2, AC-3 SOC 2 CC6.1, CC6.2
Risk/Business Impact	On a shared host or a container with broad permissions, a local attacker or co-tenant can write JSON-RPC commands to the MCP process's stdin and perform any action permitted by the PAT. The impact is confined to local access and the PAT scope, but the lack of any authentication layer is a trust-boundary violation for environments where stdio is not strictly private.
Recommendation	Document that stdio mode is intended only for single-user workstation deployments where the MCP client is the parent process and stdin/stdout are not accessible to other users. In shared/multi-user environments, use the HTTP transport with authenticated TLS and a reverse proxy. At minimum, run the stdio process under a dedicated non-root user with restrictive file permissions and consider an optional environment pre-shared key that the client must supply in the first initialize message. Secure code example: <pre># Deployment only, not code change chmod 700 /path/to/mcp-env setfacl -m u::rw /proc/<pid>/fd/0 # restrict stdin access # Prefer HTTP + mTLS for any shared host MCP_TRANSPORT=http MCP_HTTP_HOST=127.0.0.1 MCP_REQUIRE_AUTH=true MCP_API_KEY=<random-256-bit-secret></pre>
Impacted Assets	app/main.py:123-129 app/mcp/server.py:253-349
Proof Of Concept	<pre># app/main.py:123-129 def run_minimal() -> None: logger.info("Running in minimal mode (MCP server only, stdio transport)") settings = get_settings() settings.mcp.transport = "stdio" asyncio.run(main(settings=settings)) # app/mcp/server.py:253-349 (excerpt) while self.is_running: line = await loop.run_in_executor(None, sys.stdin.readline) ... response = await self._process_message(line)</pre>

V-ID 6: Unbounded resource consumption / no rate limiting

V-ID	6
Name of the Vulnerability	Unbounded resource consumption / no rate limiting
Risk	Medium Reference: 5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L)
Likelihood	Medium
Impact	Medium
Current Status	OPEN
Observation	The HTTP handler (<code>_run_http</code>) enforces only <code>client_max_size</code>/<code>max_message_size</code> on the request body. There is no per-IP rate limit, no per-client concurrency limit, no request timeout on the handler itself, and no limit on the number of concurrent GitLab API calls a single MCP request can trigger. The <code>stdio</code> loop also has no throttling beyond the max message size. A malicious or misbehaving client can therefore submit a large volume of tool calls, oversized payloads, or batched JSON-RPC requests to exhaust CPU, memory, or outbound GitLab API quota. The <code>audit.py</code> batching and flush interval are for audit writing, not request throttling. Standards Mapping: OWASP API4:2023 Unrestricted Resource Consumption CWE-770, CWE-400 NIST SP 800-53 SC-5, SC-6, SI-10 SOC 2 CC6.1, CC7.1, CC7.2
Risk/Business Impact	A single unauthenticated HTTP client can cause denial of service against the MCP server, fill logs, and consume the GitLab PAT rate limit, affecting availability for legitimate users. If the LLM client is charged per tool call or per token, this can also cause unbounded cost.

Recommendation	Add aiohttp middleware for rate limiting and a global concurrency limit on the HTTP handler. Use an in-memory token bucket keyed by IP address or API key, with configurable limits, and apply a request timeout to tool execution. Add the same limits to the CI/CD configuration and production runbooks. Secure code example: <pre># app/mcp/server.py import asyncio, time from collections import defaultdict class TokenBucket: def __init__(self, rate: float, capacity: float): self.rate = rate self.capacity = capacity self.tokens = capacity self.last = time.monotonic() self.lock = asyncio.Lock() async def acquire(self): async with self.lock: now = time.monotonic() self.tokens = min(self.capacity, self.tokens + self.rate * (now - self.last)) self.last = now if self.tokens < 1: raise web.HTTPTooManyRequests() self.tokens -= 1 _rate_limits: dict[str, TokenBucket] = defaultdict(lambda: TokenBucket(rate=10, capacity=30)) _semaphore = asyncio.Semaphore(50) async def rate_limit_middleware(request: "web.Request", handler): key = request.remote or request.headers.get("Authorization", "anonymous") await _rate_limits[key].acquire() async with _semaphore: return await handler(request)</pre>
Impacted Assets	app/mcp/server.py:391-490 (HTTP) app/mcp/server.py:253-349 (stdio)
Proof Of Concept	<pre># app/mcp/server.py:418-452 async def handle_mcp_post(request: "web.Request") -> "web.Response": if request.content_length is not None and request.content_length > max_bytes: ... try: raw_body = await request.read() payload = json.loads(raw_body.decode("utf-8")) ... result = await self.handle_jsonrpc_request(payload, headers=dict(request.headers)) ... # No rate limit, concurrency limit, or handler timeout is present.</pre>

V-ID 7: Dynamic URL path segments not URL-encoded in many services

V-ID	7
Name of the Vulnerability	Dynamic URL path segments not URL-encoded in many services
Risk	Medium Reference: Medium (manual review, 2026-09-03) - request-path/query manipulation via unencoded segments
Likelihood	Medium
Impact	Medium
Current Status	OPEN
Observation	normalize_project_id (which calls quote) is only used inconsistently across services. Dynamic segments such as file_path, branch, tag_name, slug, key, artifact_path, ref_name, commit_sha, and trigger are inserted into the endpoint path via f-strings without encoding, with the root cause in app/integrations/gitlab/client.py:_request (line 165). This can lead to request-path or query manipulation when an attacker controls a tool argument. While project_id encoding is implemented in client.py, the practice is not applied consistently or extended to other dynamic segments, which corrects the assumption that path encoding is a fully effective positive control. Standards Mapping: OWASP API8:2023 Security Misconfiguration, API10:2023 Unsafe Consumption of APIs CWE-20, CWE-116 NIST SP 800-53 SI-10 SOC 2 CC6.1
Risk/Business Impact	An attacker who controls a tool argument (for example via prompt injection or an unauthenticated call) can inject path or query fragments into the outbound GitLab API request, potentially reaching unintended endpoints, altering query parameters, or manipulating the request path beyond the intended operation.
Recommendation	Centralise URL construction so that every dynamic path segment (project_id, file_path, branch, tag_name, slug, key, artifact_path, ref_name, commit_sha, trigger, variable key, etc.) is passed through quote() consistently. Apply the encoding at the client._request boundary so no service can bypass it, and add a unit test that asserts encoding for representative segment types.
Impacted Assets	app/integrations/gitlab/client.py:165 (_request - root cause) app/integrations/gitlab/services/*.py (widespread f-string path construction)
Proof Of Concept	<pre># app/integrations/gitlab/services/*.py (representative pattern) # Dynamic segments interpolated without quote(): endpoint = f"/projects/{project_id}/repository/files/{file_path}/raw?ref={branch}" # file_path, branch, tag_name, slug, key, artifact_path, ref_name, commit_sha, trigger # are inserted via f-strings; normalize_project_id/quote() is applied inconsistently.</pre>

V-ID 8: Generic exception strings returned to MCP clients

V-ID	8
Name of the Vulnerability	Generic exception strings returned to MCP clients
Risk	Low Reference: Low-Medium (manual review, 2026-09-03) - internal error detail leakage to clients
Likelihood	Low
Impact	Low
Current Status	OPEN
Observation	Unhandled exceptions are converted with <code>str(exc) / str(e)</code> and returned in the MCP tool result or JSON-RPC error at <code>app/mcp/tools/gitlab/_base.py:143-149</code> and <code>app/mcp/server.py:585-595</code>. This can leak internal error details, exception class names, or implementation details to clients. While V-ID 3 covers header logging, this finding covers error-message leakage to the client. Standards Mapping: OWASP API8:2023 Security Misconfiguration CWE-209 NIST SP 800-53 SI-11 SOC 2 CC7.2
Risk/Business Impact	An attacker probing the MCP endpoint can gather internal implementation details, exception class names, stack-trace fragments, or file paths from returned error strings, aiding reconnaissance and the construction of more targeted attacks.
Recommendation	Return generic, client-safe error messages for unhandled exceptions and log the full detail server-side only. Map internal exceptions to a small set of stable, non-revealing error codes/messages before returning them in the tool result or JSON-RPC error object.
Impacted Assets	<code>app/mcp/tools/gitlab/_base.py:143-149</code> <code>app/mcp/server.py:585-595</code>
Proof Of Concept	<pre># app/mcp/tools/gitlab/_base.py:143-149 and app/mcp/server.py:585-595 except Exception as exc: return text_result(f"Error: {str(exc)}") # leaks internal detail # JSON-RPC path similarly returns str(e) in the error object.</pre>

V-ID 9: Tool execution audit hook defined but not wired into the registry

V-ID	9
Name of the Vulnerability	Tool execution audit hook defined but not wired into the registry
Risk	Low Reference: 3.7 (CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:L)
Likelihood	Low
Impact	Low
Current Status	OPEN

Observation	app/utils/audit.py defines audit_tool_execution() and a decorator audit_function_call() for automatic audit logging, but app/mcp/tools/registry.py:ToolRegistry.call_tool() does not invoke either helper. call_tool() logs its own execution statistics and errors via self.logger, but it does not produce a structured audit event for tool invocations. This leaves a compliance gap for forensic tracing of destructive operations (especially on the stdio transport, where no HTTP middleware audit path exists). The issue is present in both the visible category-router path and the hidden direct-tool path. Standards Mapping: OWASP API9:2023 Improper Inventory Management, API10:2023 Unsafe Consumption of APIs OWASP MCP08:2025 Lack of Audit and Telemetry CWE-778 NIST SP 800-53 AU-2, AU-3, AU-6 SOC 2 CC7.1, CC7.2, CC7.3
Risk/Business Impact	On the stdio transport, destructive or sensitive tool calls leave no structured audit trail. Compliance and incident-response teams cannot reconstruct which tools were invoked, by which identity, with which arguments, at what time. This weakens accountability and forensic analysis.
Recommendation	Wire audit_tool_execution() into ToolRegistry.call_tool(). Redact sensitive arguments before storing them (use redact_mapping from app/utils/redaction.py). Include the caller identity, transporttype, and tool name, populating user_id from the authenticated caller or the MCP request context. Secure code example: <pre># app/mcp/tools/registry.py from app.utils.audit import audit_tool_execution from app.utils.redaction import redact_mapping async def call_tool(self, name: str, arguments: Dict[str, Any], user_id: str = "unknown") -> ToolResult: ... audit_tool_execution(user_id=user_id, tool_name=name, parameters=redact_mapping(arguments), success=..., duration_ms=...,)</pre>
Impacted Assets	app/mcp/tools/registry.py:196-228 app/utils/audit.py:267-286
Proof Of Concept	<pre># app/mcp/tools/registry.py:196-228 async def call_tool(self, name: str, arguments: Dict[str, Any]) -> ToolResult: tool = self.get_tool(name) if not tool: raise ValueError(f"Tool not found: {name}") self._execution_stats[name]["total_calls"] += 1 try: result = await tool.execute(arguments) except Exception as e: self._execution_stats[name]["failed_calls"] += 1 self.logger.error(f"Failed to execute tool '{name}': {e}") raise ... # No call to audit_tool_execution() or audit_function_call() is present.</pre>

V-ID 10: Verbose validation errors disclose framework version and internal model names

V-ID	10
Name of the Vulnerability	Verbose validation errors disclose framework version and internal model names
Risk	Low Reference: Low - CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N (base 5.3; rated Low - disclosure limited to framework version and internal model names, no secrets)
Likelihood	High
Impact	Low
Current Status	OPEN
Observation	The MCP HTTP endpoint returns verbose Pydantic validation errors to the caller. A malformed tools/call (for example get_project with empty arguments) returns the internal input-model class name (GetProjectInput), the exact Pydantic version (2.13, disclosed via the errors.pydantic.dev/2.13/ documentation URL), and the input type. This reveals the validation framework and its precise version to an attacker for fingerprinting. This was confirmed live via dynamic testing against the running endpoint (/api/awgument-eqms/gitlab). The tool field name and schema are already exposed via tools/list, so the incremental disclosure here is the framework version and internal class name. As a supporting observation, the running server reported Pydantic 2.13 while uv.lock pins pydantic 2.15.0, indicating the deployed artifact was not built from the current lockfile (deployment drift). This finding is related to the generic exception-string leakage in V8, but is specific to the input-validation error path. Standards Mapping: OWASP API8:2023 Security Misconfiguration CWE-209 Information Exposure Through an Error Message, CWE-200 NIST SP 800-53 SI-11 SOC 2 CC7.2
Risk/Business Impact	An attacker can fingerprint the exact validation framework and version and confirm the input-schema shape, then look up version-specific known issues and craft more targeted attacks. Because the HTTP endpoint is unauthenticated (V1), this reconnaissance is available with no credentials. No secrets, credentials, or stack traces are exposed, so the direct impact is limited to information disclosure aiding reconnaissance.

Recommendation	Return generic, client-safe validation error messages (for example, 'Invalid input for <tool_name>') and log the full Pydantic error detail server-side only. Strip the framework documentation URL and internal model class names from responses. Consolidate this with the exception-handling hardening in V8, and rebuild the deployed image from the pinned uv.lock so the running dependency versions match the assessed lockfile. Secure code example: <pre># app/mcp/tools/gitlab/_base.py -- validation error path from pydantic import ValidationError try: req = InputModel(**arguments) except ValidationError as exc: # log full detail server-side only self.logger.info("Validation failed for %s: %s", self.name, exc) # return a generic, non-revealing message to the caller return text_result(f"Invalid input for {self.name}.")</pre>
Impacted Assets	app/mcp/tools/gitlab/_base.py:82-153 (GitLabTool.execute input validation) app/mcp/protocol/stateless_handler.py (tools/call response path) Running HTTP endpoint: POST /api/awgment-eqms/gitlab

Proof Of Concept

```
$ curl -s -X POST "http://127.0.0.1:8001/api/awgument-eqms/gitlab" \
-H "Content-Type: application/json" \
-H "x-gitlab-token: glpat-<REDACTED>" \
-d '{"jsonrpc": "2.0", "id": 2, "method": "tools/call", "params":
{"name": "get_project", "arguments": {}, "debug": true, "internal": true}}'

{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Invalid input:\n\n1 validation error for
GetProjectInput\n
      "project_id\n  Field required [type=missing,
input_value={}, "
      "input_type=dict]\n  For further information visit "
      "https://errors.pydantic.dev/2.13/v/missing"
      }
    ],
    "structuredContent": { "text": "... same message ..." },
    "isError": true
  }
}

# Disclosed: internal model class 'GetProjectInput', Pydantic version 2.13
# (via errors.pydantic.dev/2.13 URL), and input type. Confirmed live
(DAST).
```

Limitations and Exclusions

It's important to state any limitations or areas that were not fully tested.

Scope Limitations:

- The upstream GitLab instance and its native access controls
- Third-party backend datastores beyond the provided manifests
- Host and cluster infrastructure beyond the provided container/CI manifests
- Network penetration testing, and live GitLab instance testing
- The .env file and any live credentials, which were excluded from analysis

Best Practices

Adhering to the best practices outlined below enhances the overall security framework of the GitLab MCP Server while also addressing the identified vulnerabilities:

- Require authentication (API key or mTLS) and TLS on the HTTP MCP endpoint, and bind to 127.0.0.1 by default unless a container override is explicitly set.
- Add human-in-the-loop confirmation / approval tokens to all destructive tools, and disable them when the transport is unauthenticated.
- Never log raw request headers; redact credential-bearing keys and log only at DEBUG.
- Enforce max_message_size, per-IP rate limits, concurrency limits, and request timeouts on all transports before enabling TCP/WebSocket.
- Consistently URL-encode every dynamic path segment at the client._request boundary.
- Return generic, client-safe error messages and log full exception detail server-side only.
- Wire audit_tool_execution() into the tool registry with redacted arguments and caller identity.
- Keep CI/CD variable reveal and job-trace secrets behind default-deny gates; expand secret-scrub patterns and treat all GitLab-sourced content as untrusted input.
- Keep dependencies locked and scanned via a blocking pip-audit gate in CI, and periodically re-lock uv.lock to pick up security releases.

Conclusion

The GitLab MCP Server currently has Critical, High, Medium, and Low vulnerabilities.

- It is advisable to prioritize the resolution of the identified critical and high vulnerabilities in the GitLab MCP Server to enhance overall security measures and mitigate related risks.
- We recommend conducting a re-test or follow-up assessment once the critical and high vulnerabilities have been addressed.

Disclaimer

Standard legal disclaimer for the VAPT report.

- This report and its contents are confidential and should be shared only with authorized personnel.
- The findings are based on the agreed-upon scope and methodology.
- This report reflects that the security posture of the target MCP server at the time of testing is not a guarantee against all future attacks. New vulnerabilities may emerge, and configurations may change.

---- End of Document ----